# MANUAL TÉCNICO DEL DESARROLLADOR

*Librería Virtual | MF0227_3 · UF2405 · UD2*

MANUAL TÉCNICO DEL DESARROLLADOR Sistema de Gestión de Catálogo - Librería Virtual (J2EE)

Módulo Formativo: MF0227_3 - Programación orientada a objetos Unidad Formativa: UF2405 - Modelo de programación web y bases de datos Actividad: UD2 - Capas de una aplicación web Versión: 1.0 | Fecha: Octubre 2026 Clasificación: Documento técnico interno

**ÍNDICE**

1. Introducción y Objetivos

2. Arquitectura del Sistema

3. Stack Tecnológico

4. Estructura del Proyecto

5. Descripción de Componentes y Código

6. Flujo de una Petición HTTP

7. Base de Datos H2

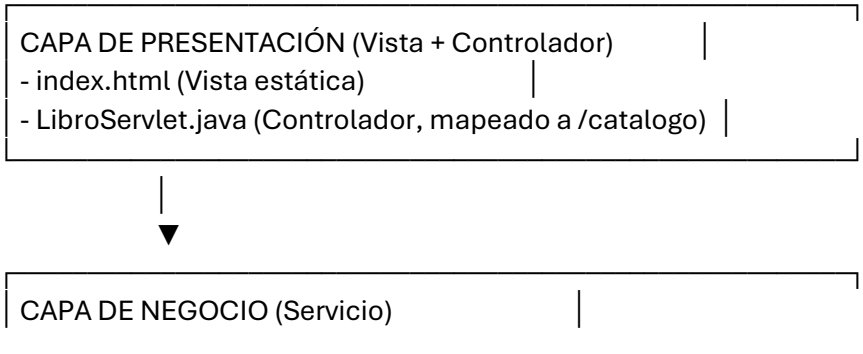
8. Guía de Extensión y Mantenimiento

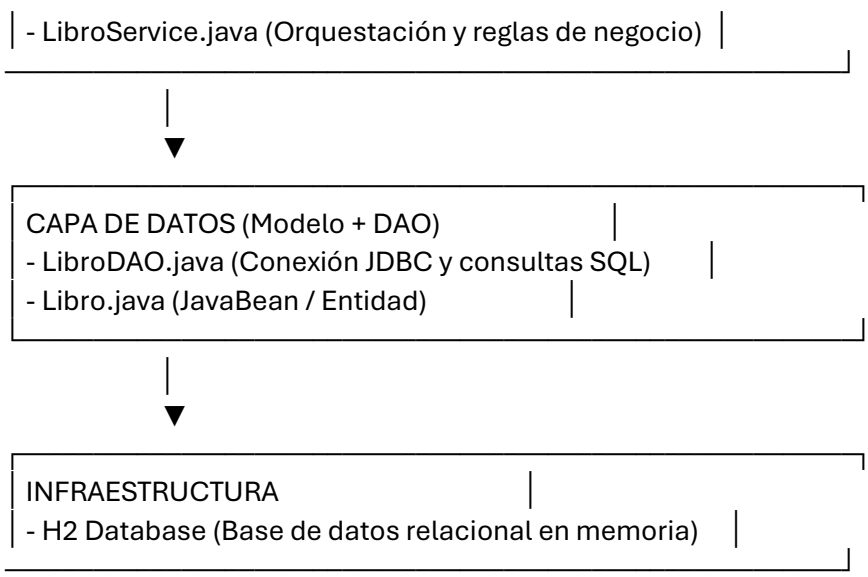
**1. INTRODUCCIÓN Y OBJETIVOS**

Este manual técnico está dirigido a desarrolladores que necesiten comprender, mantener o extender la aplicación web "Librería Virtual". Documenta la arquitectura, decisiones de diseño y mecanismos de despliegue del sistema, cumpliendo con los criterios de evaluación de la UF2405 sobre arquitectura multicapa y aislamiento del entorno.

**2. ARQUITECTURA DEL SISTEMA**

La aplicación implementa el patrón Modelo-Vista-Controlador (MVC) distribuido en 3 capas lógicas:






### 3. STACK TECNOLÓGICO

| Tecnología | Versión | Función |
|---|---|---|
| Java JDK | 8+ (Compatible con 25) | Lenguaje de programación |
| Apache Maven | 3.9.6 | Gestión de dependencias y build |
| Apache Tomcat | 7.0.47 (Embebido) | Servidor de aplicaciones |
| H2 Database | 2.2.224 | Base de datos en memoria |
| Servlet API | 3.1.0 | API de servlets Java EE |

### 4. ESTRUCTURA DEL PROYECTO (Estándar Maven)

```
proyecto-libreria/
├── pom.xml                    # Descriptor Maven
├── lanzar_libreria.bat        # Script de ejecución
├── tools/                     # Maven local (no afecta SO)
└── src/
    └── main/
        ├── java/
        │   └── com/empresa/libreria/
        │       ├── model/Libro.java       # Entidad
        │       ├── dao/LibroDAO.java      # Acceso a datos
        │       ├── service/LibroService.java# Lógica de negocio
        │       └── web/LibroServlet.java  # Controlador
        └── webapp/
            ├── index.html         # Página principal
            └── css/estilos.css    # Hoja de estilos
```

### 5. DESCRIPCIÓN DE COMPONENTES

### 5.1. `pom.xml` (Configuración Maven)

Define el proyecto, las dependencias (Servlet API con scope `provided`, y H2 Database) y el plugin `tomcat7-maven-plugin` que permite levantar el servidor embebido con el comando `mvn tomcat7:run`.

### 5.2. `model/Libro.java` (JavaBean)

Clase que representa la entidad del dominio. Implementa `Serializable` y encapsula las propiedades `id`, `titulo`, `autor` y `precio` mediante getters.

### 5.3. `dao/LibroDAO.java` (Acceso a Datos)

Encapsula la lógica de acceso a la base de datos. Utiliza la API JDBC para conectar con H2. Característica destacada: Utiliza la cláusula `INIT=` en la URL de conexión para crear la tabla `libros` e insertar datos de prueba automáticamente en memoria al establecer la conexión, eliminando la necesidad de scripts SQL externos.

### 5.4. `service/LibroService.java` (Lógica de Negocio)

Actúa como intermediario. Recibe las peticiones del Servlet y llama al DAO. En un entorno de producción, aquí se implementarían validaciones complejas, cálculos de impuestos o reglas de descuento.

### 5.5. `web/LibroServlet.java` (Controlador)

Servlet anotado con `@WebServlet("/catalogo")`. Extiende de `HttpServlet` y sobrescribe el método `doGet()`. Utiliza `PrintWriter` para generar HTML dinámico inyectando los datos de la lista de libros recibida del Service.

## 6. FLUJO DE UNA PETICIÓN HTTP

1. El navegador solicita `GET /libreria/catalogo`.

2. Tomcat enruta la petición al `LibroServlet`.

3. El Servlet instancia `LibroService` y llama a `listarLibros()`.

4. El Service llama a `LibroDAO.obtenerCatalogo()`.

5. El DAO ejecuta `SELECT * FROM libros` vía JDBC contra H2.

6. El DAO mapea el `ResultSet` a una `List<Libro>` y la devuelve.

7. El Servlet itera sobre la lista y genera el código HTML de la tabla.

8. Tomcat envía la respuesta HTTP 200 con el HTML al navegador.

## 7. BASE DE DATOS H2

Se utiliza H2 en modo in-memory (`jdbc:h2:mem:libreria`).

Ventaja: No requiere instalación, es extremadamente rápida y se reinicia automáticamente al detener el servidor, garantizando un entorno de pruebas limpio en cada ejecución.

URL de conexión: `jdbc:h2:mem:libreria;DB_CLOSE_DELAY=-1;INIT=CREATE TABLE...`

## 8. GUÍA DE EXTENSIÓN Y MANTENIMIENTO

### 8.1. Cómo añadir un nuevo endpoint (Ejemplo: Crear Libro)

1. DAO: Añadir método `crearLibro(Libro l)` con un `INSERT INTO`.

2. Service: Añadir método `registrarLibro(Libro l)` con validaciones de negocio.

3. Servlet: Sobrescribir el método `doPost()`, leer los parámetros del `HttpServletRequest`, llamar al Service y redirigir al catálogo.

### 8.2. Compilación y Empaquetado Manual

Si no se utilizan los scripts `.bat`, el proyecto puede compilarse desde la terminal:

```
cd proyecto-libreria
set "PATH=%PATH%;%cd%\tools\apache-maven-3.9.6\bin"
call mvn clean compile tomcat7:run
```

### 8.3. Entrega y Despliegue

Para entregar el código a otros desarrolladores o al evaluador, se debe generar un archivo ZIP excluyendo las carpetas `target/` (compilados) y `tools/` (Maven local). El receptor solo necesitará tener Java instalado y ejecutar los scripts `.bat` para regenerar su propio entorno aislado.